

Microkernels

Definition

A **microkernel** is an operating-system design in which only a very small core runs in **kernel mode**, while most other operating-system components run as separate **user-mode processes**. The goal is to increase reliability, modularity, and fault isolation.

1. Basic Idea

Main Concept

In a microkernel design:

- the kernel is kept as small as possible,
- only essential functions remain in kernel mode,
- file systems, drivers, and many services run in user mode,
- faults in one component are less likely to crash the whole system.

Reason for the Design

Bugs inside the kernel are very dangerous because they can crash the entire operating system immediately. User processes have less power, so bugs there are usually less destructive.

2. Why Smaller Kernel is Safer

Bug Problem

Large software systems usually contain many bugs. Therefore, if the kernel is huge, the chance of serious kernel bugs also becomes high.

Effect

In a monolithic system:

- many drivers and subsystems run inside the kernel,
- one bad driver may crash the entire machine.

In a microkernel system:

- many components run as isolated user processes,
- failure of one component usually affects only that component.

Example

A buggy audio driver in a monolithic system may crash the whole computer. In a microkernel system, the same buggy audio driver may only stop or distort sound, while the rest of the system continues running.

3. Main Features of Microkernels

Core Characteristics

A microkernel system usually has:

- a very small kernel,
- separate user-mode drivers,
- separate user-mode file systems,
- message passing between processes,
- strong isolation and protection.

4. Common Use of Microkernels

Where They Are Used

Microkernels are especially common in systems that need very high reliability.

Typical Application Areas

They are widely used in:

- real-time systems,
- industrial systems,
- avionics,
- military systems,
- other mission-critical environments.

Examples

Some well-known microkernel-based systems include:

- Integrity
- K42
- L4
- PikeOS
- QNX
- Symbian
- MINIX 3

5. MINIX 3 as an Example

MINIX 3

MINIX 3 is a POSIX-conformant open-source microkernel system. It pushes modularity very far by moving most operating-system functionality into independent user-mode processes.

Microkernel Size

The MINIX 3 microkernel is very small compared with full monolithic kernels. Its main responsibilities include:

- process management,
- scheduling,
- interrupt handling,
- interprocess communication,
- a small set of kernel calls.

6. What the Microkernel Does

Microkernel Functions

The microkernel itself performs only the most essential tasks, such as:

- catching interrupts,
- switching processes,
- scheduling,
- passing messages between processes,
- handling low-level kernel calls.

Kernel Calls

These kernel calls support operations like:

- attaching interrupt handlers,
- moving data between address spaces,
- installing memory maps for new processes.

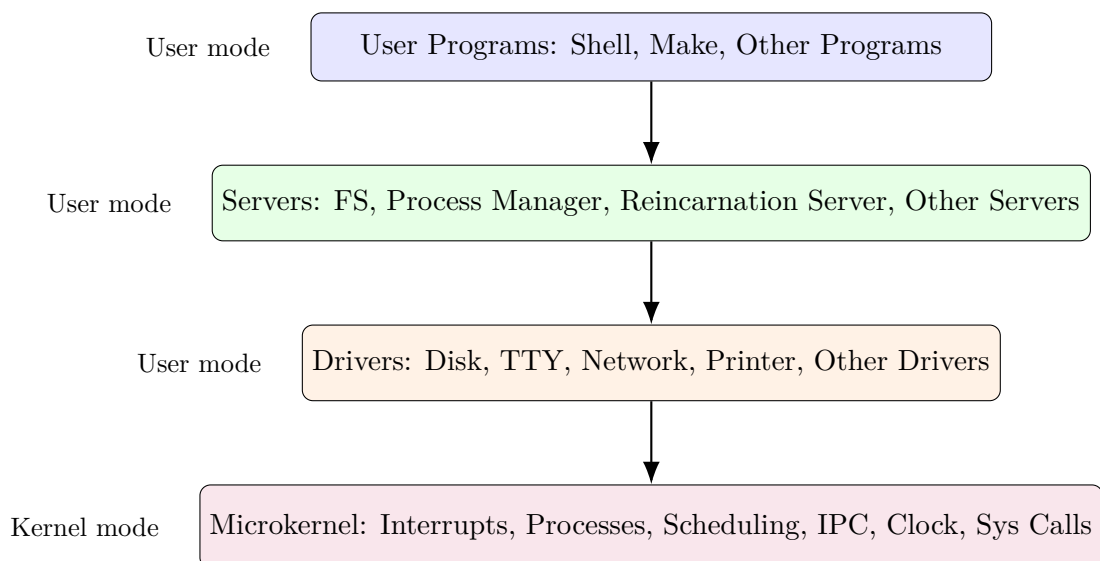
7. Process Structure of MINIX 3

Layered User-Mode Structure

Outside the microkernel, MINIX 3 has several user-mode layers:

- device drivers,
- servers,
- user programs.

7.1 Simplified MINIX 3 Structure



8. Device Drivers in User Mode

Drivers

In MINIX 3, most device drivers run as separate user-mode processes, not inside the kernel.

Restriction

Because drivers run in user mode:

- they cannot directly access hardware I/O ports,
- they must ask the kernel to perform low-level I/O operations,
- the kernel checks whether the driver is authorized.

Benefit

This protection prevents one buggy driver from damaging unrelated parts of the system. For example, a faulty audio driver cannot accidentally write on the disk.

9. Servers in User Mode

Servers

Above the drivers is a layer of user-mode **servers**. These servers perform most of the higher-level work of the operating system.

Examples of Servers

Examples include:

- file servers for file systems,
- the process manager,
- the reincarnation server,
- other service processes.

Service Request

If a user process wants to perform a POSIX operation such as **read**, it sends a short message to the appropriate server asking for that service.

10. Interprocess Communication

IPC

A central mechanism in microkernels is **interprocess communication (IPC)**. Processes cooperate by sending messages to each other.

Why IPC Matters

Since drivers, servers, and user programs are separated into different processes, message passing is needed to:

- request services,
- transfer information,
- coordinate system actions.

11. Reincarnation Server

Special Server

One important MINIX 3 server is the **reincarnation server**.

Its Job

The reincarnation server:

- monitors drivers and servers,
- checks whether they are functioning correctly,
- restarts faulty components automatically.

Self-Healing

Because of this mechanism, the system can recover from some failures automatically, without user intervention. This improves reliability and makes the system **self-healing**.

12. Restrictions and Protection

Controlled Privileges

Microkernel systems place strict limits on what each process can do.

Examples of Restrictions

Restrictions may control:

- which I/O ports a driver may access,
- which kernel calls a process may use,
- which processes may send messages to others,
- when one process may access another process's address space.

Result

Each component gets only the power required for its own work and nothing more. This greatly reduces the damage that a buggy component can cause.

13. Policy and Mechanism

Important Design Principle

A related idea is to keep the **mechanism** in the kernel, but move the **policy** outside the kernel whenever possible.

Meaning

For example, in process scheduling:

- the **mechanism** is the actual act of selecting and running a process,
- the **policy** is the rule for deciding which process should get priority.

Scheduling Example

The kernel may provide the mechanism to run the highest-priority runnable process. A user-mode component may provide the policy by deciding what priorities the processes should have.

Advantage

Separating policy from mechanism makes the kernel smaller, cleaner, and more flexible.

14. Advantages of Microkernels

Main Advantages

Microkernels provide:

- higher reliability,
- better fault isolation,
- stronger modularity,
- easier recovery from component failures,
- improved security and protection.

15. Possible Drawback

Possible Limitation

Because many services are moved out of the kernel and communicate through messages, microkernels may involve more communication overhead than monolithic kernels.

16. Summary Points

Quick Review

A microkernel:

- keeps only essential functions in kernel mode,
- moves many services to user mode,
- uses message passing for communication,
- isolates faults better than monolithic systems,
- is especially useful in high-reliability systems.